New four-dimensional linear codes over $\mathbb{F}_8$ and $\mathbb{F}_9$ improving Maruta's tables

Claude (AI model by Anthropic)
run and submitted by Aleksandr Bersenev

October 8, 2026
Produced autonomously by Claude (Opus 5.5) agents in Claude Code; run and submitted by Aleksandr Bersenev.
Not an official Anthropic publication. Not peer reviewed.

Abstract

Let $n_q(4,d)$ be the smallest length of a linear $[n,4,d]_q$ code. We give 32 explicit four-dimensional linear codes, 24 over $\mathbb{F}_9$ and 8 over $\mathbb{F}_8$. Together with the codes obtained from them by puncturing, they are shorter than the upper bound recorded in Maruta's online tables of $n_q(4,d)$ (as of October 8, 2026) for 191 values of d: for 149 values of d with $190 \le d \le 513$ when $q = 9$ (twenty of them by 2), and for 42 values of d with $137 \le d \le 376$ when $q = 8$. In 29 cases the new length equals the lower bound proved in the literature, which gives $n_8(4,d) = g_8(4,d)+1$ for $d \in \{337,\dots,345\} \cup \{353,354\} \cup \{361,\dots,376\}$ and $n_9(4,512) = 578$, $n_9(4,513) = 579$, where $g_q(4,d)$ is the Griesmer bound. The codes were found by a prescribed-automorphism search (orbits of cyclic subgroups of $\mathrm{PGL}(4,q)$, integer programming, and greedy lengthening). The existence of every code is certified in Lean 4 by a kernel-checked computation whose soundness is proved in Lean. The lower bounds are cited, not re-proved.

Contents

1 Introduction **2**

2 Definitions and main results **3**

3 The construction **4**
3.1 Geometric reformulation 4
3.2 Prescribed automorphisms and an integer program 4
3.3 Greedy lengthening 4
3.4 The base codes 5
3.5 Three examples 5

4 Verification **6**
4.1 Exhaustive enumeration 6
4.2 Lean formalization 6

5 Limitations and open questions **8**

A All improved table entries **9**

B Data and reproduction **11**

1 Introduction

For a prime power q and integers $k, d \geq 1$, let $n_q(k,d)$ denote the minimum length n for which a linear $[n,k,d]_q$ code exists, i.e. a k-dimensional subspace of $\mathbb{F}_q^n$ all of whose nonzero vectors have Hamming weight at least d. Determining $n_q(k,d)$ is one of the basic problems of coding theory (see the surveys of Hill [7]). The classical lower bound is the Griesmer bound [5, 16]

$$n_q(k,d) \;\geq\; g_q(k,d) \;:=\; \sum_{i=0}^{k-1} \left\lceil \frac{d}{q^i} \right\rceil .$$

For $k \leq 3$ and small q the function $n_q(k,d)$ is known. For $k = 4$ it is known for $q \leq 4$, but for $q \geq 5$ many values remain open. T. Maruta maintains online tables [14] with the current intervals $n_{\mathrm{lo}}(d) \leq n_q(4,d) \leq n_{\mathrm{up}}(d)$ for $q \in \{5,7,8,9\}$ and all d below the range where $n_q(4,d) = g_q(4,d)$ is known.

- For $q = 8$ the table follows Kanazawa–Maruta [9], with later updates (e.g. [1, 12]) for values of d not considered here. That paper determines $n_8(4,d)$ for 117 values of d with $113 \leq d \leq 832$ and gives bounds for the others. In particular [9, Thm. 1.1(4)] shows $g_8(4,d)+1 \leq n_8(4,d) \leq g_8(4,d)+2$ for $337 \leq d \leq 376$.
- For $q = 9$ the table follows Kumegawa–Okazaki–Maruta [11] (a few lower bounds have been updated since), who constructed many mid-range codes "with the aid of a computer, but the upper bounds are still weak to be improved" [11, p. 10]. In particular [11, Thm. 1.7(4)] gives $g_9(4,d)+1 \leq n_9(4,d) \leq g_9(4,d)+2$ for $d = 512, 513$.

In the mid range the intervals are typically $g_9(4,d) \leq n_9(4,d) \leq g_9(4,d)+3$ or $+4$ for $q = 9$, and $g_8(4,d)+1 \leq n_8(4,d) \leq g_8(4,d)+2$ for $q = 8$. These lengths ($n \geq 160$) lie outside Grassl's code tables [4], which stop at $n = 130$ for $q = 8, 9$. They also lie outside MinT [15], whose linear-code data are limited to co-dimension at most 173 for base 8 and 150 for base 9.

What is new. We construct 32 linear codes with $k = 4$ over $\mathbb{F}_8$ and $\mathbb{F}_9$. Together with their punctured codes, they lower the recorded upper bound $n_{\mathrm{up}}(d)$ for 191 table entries (Theorems 2.3 and 2.4). Twenty-nine of the new lengths coincide with lower bounds proved in [9, 11], which determines these values of $n_q(4,d)$ (Corollary 2.5). To our knowledge, as of October 8, 2026, none of these codes or parameters appeared before. We compared against Maruta's table, the two papers above, Grassl's tables and MinT, and searched the recent literature on $n_8(4,d)$ and $n_9(4,d)$ [1, 12, 10]. For every improved entry with $q = 8$ the table row (lower and upper bound) is the one of Table 3 of [9], and for every improved entry with $q = 9$ the upper bound is the one of Table 2 of [11]; the updates of the table since then concern other values of d or only lower bounds. The most recent nonexistence results [10, Thm. 1.3] ($n_8(4,d) = g_8(4,d)+1$ for $d \in \{278, 398, 406\}$, $n_9(4,d) = g_9(4,d)+1$ for $d \in \{583, 808\}$, and $n_9(4,196) \geq g_9(4,196)+1 = 223$) are not yet reflected in the table; they do not affect the entries below, except that the lower bound for $d = 196$, $q = 9$, should now be read as 223 instead of 222. We cannot rule out unpublished constructions.

Method in one paragraph. By the standard correspondence [6], an $[n, 4, d]_q$ code is a multiset of n points of $\mathrm{PG}(3, q)$ meeting every plane in at most $n - d$ points. Following the prescribed-automorphism approach of Braun–Kohnert–Wassermann [2] and the "orbits of projectivities" of Maruta–Shinohara–Takenaka [13], we chose random projectivities T of moderate order. We computed the orbits of $\langle T \rangle$ on points and on planes, and solved with HiGHS [8] the integer program "choose a union of point orbits of maximum size subject to all plane intersections being $\leq r$". We then lengthened the solution greedily by single points. Every hit was checked by exhaustive enumeration of all $q^4 - 1$ nonzero codewords, and all base codes were then certified in Lean 4 [3].

2 Definitions and main results

Definition 2.1. Let $L = (c_1, \ldots, c_n)$ be a list of vectors of $\mathbb{F}_q^4$ (the columns of a $4 \times n$ matrix G). For $u \in \mathbb{F}_q^4$ the codeword of u is $uG = (u \cdot c_1, \ldots, u \cdot c_n)$. We say that L *generates an* $[n, 4, \geq d]_q$ *code* if $\mathrm{wt}(uG) \geq d$ for all $u \neq 0$. Since $d \geq 1$, the map $u \mapsto uG$ is then injective, so the row space of G is a linear code of length n, dimension exactly 4 and minimum distance at least d. Columns may repeat (non-projective codes), as in the definition of $n_q(k, d)$.

Lemma 2.2 (puncturing)**.** *If L generates an $[n, 4, \geq d]_q$ code and $0 \leq t < d$, then deleting any t columns gives an $[n - t, 4, \geq d - t]_q$ code. Hence $n_q(4, d - t) \leq n_q(4, d) - t$.*

Proof. Deleting t coordinates lowers every weight by at most t. Since $d - t \geq 1$, injectivity is preserved. □

Theorem 2.3 ($q = 9$)**.** *For each of the* 149 *values of d in Table 3, there is a linear $[n, 4, \geq d]_9$ code whose length n (column "new") is strictly smaller than the upper bound $n_{\mathrm{up}}(d)$ recorded in [14]. The improved values of d form the ranges* 190–192, 194–200, 231–232, 241–243, 253–256, 262–279, 289–292, 298–312, 333–360, 370–375, 379–387, 397–417, 424–429, 433–438, 487–489, 496–500, 505–513. *For twenty of them the improvement is by* 2, *for example* $[312, 4, 274]_9$ *(previous upper bound* 314*)*, $[345, 4, 303]_9$ *(*347*)*, $[380, 4, 334]_9$ *(*382*)*, $[390, 4, 343]_9$ *(*392*)*, $[420, 4, 370]_9$ *(*422*)*, $[460, 4, 406]_9$ *(*462*)*, $[468, 4, 414]_9$ *(*470*)*.

Theorem 2.4 ($q = 8$)**.** *For each of the* 42 *values of d in Table 2, namely* 137–140, 221–224, 329–345, 353–354 *and* 361–376, *there is a linear $[n, 4, \geq d]_8$ code with n strictly smaller than $n_{\mathrm{up}}(d)$ in [14].*

Corollary 2.5. *Combining Theorems 2.3 and 2.4 with the lower bounds of [9, Thm. 1.1(4)] and [11, Thm. 1.7(4)] gives*

$$n_8(4, d) = g_8(4, d) + 1 \quad \text{for } d \in \{337, \ldots, 345\} \cup \{353, 354\} \cup \{361, \ldots, 376\},$$

and $n_9(4, 512) = 578 = g_9(4, 512) + 1$, $n_9(4, 513) = 579 = g_9(4, 513) + 1$.

Proof. By the cited theorems, $n \geq g + 1$ for these d. For the upper bound we have explicit codes:

- $[395, 4, 344]_8$ and its punctured codes $[395 - t, 4, 344 - t]_8$ for $t \leq 7$ cover $d = 337, \ldots, 344$;
- $[397, 4, 345]_8$ covers $d = 345$;
- $[407, 4, 354]_8$ and its punctured code $[406, 4, 353]_8$ cover $d = 353, 354$;
- $[422, 4, 368]_8$ and its punctured codes cover $d = 361, \ldots, 368$;

- $[431, 4, 376]_8$ and its punctured codes cover $d = 369, \ldots, 376$;
- $[579, 4, 513]_9$ and $[578, 4, 512]_9$ cover the two values for $q = 9$.

In each case the length equals $g_q(4, d) + 1$, which a direct computation confirms; for example $g_8(4, 344) = 344 + 43 + 6 + 1 = 394$. □

The lower bounds used in Corollary 2.5 are *not* re-proved here; the Lean development certifies only the existence of the codes (Section 4).

3 The construction

3.1 Geometric reformulation

Identify a nonzero column $c \in \mathbb{F}_q^4$ with the point $\langle c \rangle$ of $\mathrm{PG}(3, q)$ and a nonzero message u with the plane $u^\perp = \{\langle x \rangle : u \cdot x = 0\}$. If M is the multiset of columns, then $\mathrm{wt}(uG) = n - |M \cap u^\perp|$. So L generates an $[n, 4, \geq n - r]_q$ code iff every plane contains at most r points of M, counted with multiplicity. $\mathrm{PG}(3, q)$ has $\theta = q^3 + q^2 + q + 1$ points and planes, i.e. 585 for $q = 8$ and 820 for $q = 9$.

3.2 Prescribed automorphisms and an integer program

Let $T \in \mathrm{GL}(4, q)$, and let $\langle T \rangle$ act on points by $\langle c \rangle \mapsto \langle Tc \rangle$ and on planes by $u^\perp \mapsto (uT^{-1})^\perp$. If M is a union of point orbits $O_1, \ldots, O_s$ with multiplicities x_i, then the plane intersection numbers are constant on plane orbits. Let $H_1, \ldots, H_t$ be representatives of the plane orbits and $a_{ji} = |O_i \cap H_j|$. Then

$$\max \sum_i |O_i|\, x_i \quad \text{s.t.} \quad \sum_i a_{ji} x_i \leq r \ (1 \leq j \leq t), \qquad x_i \in \{0, 1\}$$

yields a point set with all plane intersections $\leq r$, i.e. an $[n, 4, \geq n - r]_q$ code with $n = \sum |O_i| x_i$. For a target entry $(d, n_{\mathrm{up}}(d))$ of the table, the code is an improvement iff $n < n_{\mathrm{up}}(n - r)$.

We sampled random $T \in \mathrm{GL}(4, q)$ whose projective order lies in a prescribed set. Orders $8, 16, 20, 24, 26, 40$ were productive for $q = 9$ and $7, 14, 21$ for $q = 8$; these give between 33 and 108 point orbits. We solved the program with HiGHS (single thread, 4–12 s per instance, not necessarily to optimality) for every r with an open target. In total about 370 groups were tried. The search scripts are not part of this package; the resulting codes and the verification scripts are (Appendix B).

3.3 Greedy lengthening

If M has maximum plane intersection r and P is a point all of whose planes contain at most $r - 1$ points of M, then $M + P$ (multiplicities allowed) still has maximum plane intersection r. This lengthens the code from $[n, 4, d]$ to $[n + 1, 4, d + 1]$. Applying this step greedily to the orbit solutions produced 13 of the 32 base codes (marked "orbits + lengthening" in Table 1); some of them have repeated columns.

3.4 The base codes

Table 1 lists the 32 base codes. Every improved entry of Tables 2 and 3 is a punctured code of one of them. The base codes were chosen by a greedy set cover among the 230 verified code files. The table shows the construction (order of the prescribed cyclic group, when the code is an orbit union), the maximum column multiplicity, the number of distinct nonzero weights, the gcd of the nonzero weights, the maximum weight, the number of table entries the code improves (in parentheses: how many of them meet the lower bound), and the SHA-256 prefix of the data file `data/codes/q<q>_n<n>_d<d>.txt`.

Table 1: The 32 base codes (all certified in Lean).

code	construction	mult	#wts	gcd	$w_{\max}$	entries	SHA-256 (prefix)
$[163, 4, 140]_8$	$\lvert\langle T\rangle\rvert = 14$	1	9	2	162	4	`ab8a14270b42d5f4`
$[259, 4, 224]_8$	$\lvert\langle T\rangle\rvert = 14$	1	6	4	252	4	`a8564ba5338ec784`
$[385, 4, 335]_8$	$\lvert\langle T\rangle\rvert = 14$	1	4	1	384	7	`324fd0eaa554cb6b`
$[395, 4, 344]_8$	$\lvert\langle T\rangle\rvert = 14$	1	3	8	392	8 (8)	`57f2c76cc0bbb19c`
$[397, 4, 345]_8$	$\lvert\langle T\rangle\rvert = 14$	1	8	1	394	1 (1)	`cd6dad28e828a03d`
$[407, 4, 354]_8$	$\lvert\langle T\rangle\rvert = 7$	1	8	2	406	2 (2)	`69103d3f0cced90e`
$[422, 4, 368]_8$	$\lvert\langle T\rangle\rvert = 7$	1	4	8	400	8 (8)	`fe9075506fa4377f`
$[431, 4, 376]_8$	$\lvert\langle T\rangle\rvert = 7$	1	4	8	400	8 (8)	`d8e339c3b2489a27`
$[220, 4, 192]_9$	$\lvert\langle T\rangle\rvert = 20$	1	5	3	204	3	`392a4a3c89c5820e`
$[229, 4, 200]_9$	orbits + lengthening	2	12	1	226	7	`08f00c7af5ce8ec9`
$[265, 4, 232]_9$	$\lvert\langle T\rangle\rvert = 24$	1	13	1	260	2	`b6b70d374a2836cd`
$[277, 4, 243]_9$	$\lvert\langle T\rangle\rvert = 24$	1	8	3	273	3	`caca63a3daca3d84`
$[292, 4, 256]_9$	orbits + lengthening	2	12	1	290	4	`72e035ddbf489fdc`
$[303, 4, 266]_9$	orbits + lengthening	2	10	1	301	5	`13ee0205379bb524`
$[314, 4, 276]_9$	orbits + lengthening	2	11	1	312	10	`d62a7353317e2154`
$[318, 4, 279]_9$	orbits + lengthening	3	6	3	315	3	`ecc272c062828ccf`
$[333, 4, 292]_9$	$\lvert\langle T\rangle\rvert = 24$	1	16	1	333	4	`d7da904ca001abf6`
$[345, 4, 303]_9$	orbits + lengthening	2	8	3	342	6	`93b8f5fd281d85f9`
$[355, 4, 312]_9$	orbits + lengthening	3	7	3	351	9	`c30d4cf4cead8680`
$[382, 4, 336]_9$	orbits + lengthening	2	7	3	360	4	`636b151450aa9b01`
$[390, 4, 343]_9$	orbits + lengthening	1	13	1	385	7	`a735efecc63eee1b`
$[408, 4, 360]_9$	$\lvert\langle T\rangle\rvert = 16$	1	3	9	378	17	`17659a7cc1f6e578`
$[422, 4, 372]_9$	orbits + lengthening	2	7	3	420	3	`f9aae5b910d2ada1`
$[426, 4, 375]_9$	orbits + lengthening	1	14	1	425	3	`7cb0da2406c65270`
$[439, 4, 387]_9$	orbits + lengthening	3	3	9	432	9	`3d90eb9cf624b466`
$[468, 4, 414]_9$	$\lvert\langle T\rangle\rvert = 26$	1	4	9	468	18	`b20bdceedfbef563`
$[473, 4, 417]_9$	orbits + lengthening	2	7	3	456	3	`f62b5bdae030557d`
$[486, 4, 429]_9$	$\lvert\langle T\rangle\rvert = 16$	1	6	3	453	6	`c3d4ea832d31fa01`
$[496, 4, 438]_9$	$\lvert\langle T\rangle\rvert = 8$	1	9	3	480	6	`f7f0d15a44168321`
$[553, 4, 489]_9$	$\lvert\langle T\rangle\rvert = 24$	1	8	1	552	3	`8e9a265ada7c3941`
$[565, 4, 500]_9$	$\lvert\langle T\rangle\rvert = 24$	1	6	1	528	5	`83417062ebf18cb2`
$[579, 4, 513]_9$	$\lvert\langle T\rangle\rvert = 16$	1	3	9	576	9 (2)	`4e272d7d4ec1953b`

3.5 Three examples

Field elements are written as integers. In $\mathbb{F}_8 = \mathbb{F}_2[x]/(x^3 + x + 1)$, the integer $c_0 + 2c_1 + 4c_2$ denotes $c_0 + c_1 x + c_2 x^2$. In $\mathbb{F}_9 = \mathbb{F}_3[x]/(x^2 + x + 2)$, the integer $c_0 + 3c_1$ denotes $c_0 + c_1 x$. The matrices act on column vectors.

- $[431, 4, 376]_8$ is a union of 4 fixed points and 61 orbits of size 7 of $T = \begin{pmatrix} 3&0&3&7\\7&3&3&1\\0&2&6&5\\6&2&6&6 \end{pmatrix}$ (projective order 7). Its weight enumerator is $376^{3577}\,384^{441}\,392^{49}\,400^{28}$ (exponents are the numbers of codewords), so the code is 8-divisible.
- $[395, 4, 344]_8$ is a union of orbits of sizes $1, 2$ and 28×14 of $T = \begin{pmatrix} 2&5&5&6\\2&4&0&0\\6&0&6&7\\4&6&0&0 \end{pmatrix}$ (order 14). Its weight enumerator is $344^{3255}\,352^{833}\,392^{7}$, again 8-divisible. The code $[422, 4, 368]_8$ (order 7) has weights $368, 376, 392, 400$.
- $[579, 4, 513]_9$ is a union of orbits of sizes $1, 2$ and 36×16 of $T = \begin{pmatrix} 7&4&5&5\\4&1&8&3\\7&4&5&2\\6&5&6&1 \end{pmatrix}$ (order 16). Its weight enumerator is $513^{5336}\,522^{1216}\,576^{8}$, so the code is 9-divisible.

We have no conceptual explanation of these divisible codes. They would be natural starting points for a geometric (e.g. minihyper-based) description.

4 Verification

4.1 Exhaustive enumeration

For every code file, a short Python program (`data/verify.py`) enumerates all $q^4 - 1$ nonzero messages, computes the codewords with table-based $\mathbb{F}_q$ arithmetic, and prints the minimum distance and the full weight distribution. All 32 base codes, and all 230 code files of the search, have the claimed minimum distance. An independent re-implementation with its own $\mathbb{F}_8/\mathbb{F}_9$ arithmetic confirmed all 230 files. We also recomputed the minimum distances directly from the column lists embedded in the Lean file, in the field representation used there.

4.2 Lean formalization

The file `lean/Main.lean` is self-contained Lean 4.34.1 (core library only, no Mathlib), with one namespace per field. It is checked by running `lean Main.lean` in the directory `lean/` with the Lean 4.34.1 toolchain (no project file and no Mathlib are needed).

Fields. $\mathbb{F}_9$ is realised as $\mathbb{F}_3[i]/(i^2+1)$ (the polynomial x^2+1 is irreducible over $\mathbb{F}_3$). The codes were found over $\mathbb{F}_3[x]/(x^2+x+2)$ and were transported by the field isomorphism $x \mapsto 1+i$, which preserves Hamming weights. $\mathbb{F}_8$ is realised as $\mathbb{F}_2[x]/(x^3+x+1)$ with three Boolean coefficients.

```
structure GF9 where
  re : Fin 3
  im : Fin 3
instance : Add GF9 := ⟨fun x y => ⟨x.re + y.re, x.im + y.im⟩⟩
instance : Mul GF9 := ⟨fun x y => ⟨x.re * y.re - x.im * y.im, x.re * y.im + x.im * y.re
    ⟩⟩
```

The theorems `Q9.GF9_is_field` and `Q8.GF8_is_field` state, and prove by exhaustive `decide`, associativity and commutativity of $+$ and $\cdot$, the neutral elements, additive inverses, distributivity, $1 \neq 0$, and $\forall x \neq 0\ \exists y,\ xy = 1$.

The statement. The predicate used in the main theorems is, verbatim (namespace `Q9`; `Q8` is identical with `GF8`):

```
abbrev Vec4 := GF9 × GF9 × GF9 × GF9
def dot (u c : Vec4) : GF9 := u.1 * c.1 + u.2.1 * c.2.1 + u.2.2.1 * c.2.2.1 + u.2.2.2
    * c.2.2.2
def codeword (L : List Vec4) (u : Vec4) : List GF9 := L.map (dot u)
def wt (v : List GF9) : Nat := v.countP (fun x => decide (x ≠ 0))
def IsCode (L : List Vec4) (n d : Nat) : Prop :=
  L.length = n ∧ ∀ u : Vec4, u ≠ (0, 0, 0, 0) → d ≤ wt (codeword L u)
```

The main theorems are long conjunctions, one conjunct per improved table entry:

```
theorem Q9.main9 :
    (∃ L : List Vec4, IsCode L 218 190) ∧ (∃ L : List Vec4, IsCode L 219 191) ∧ ...
    ∧ (∃ L : List Vec4, IsCode L 579 513)                  -- 149 conjuncts
theorem Q8.main8 :
    (∃ L : List Vec4, IsCode L 160 137) ∧ ... ∧ (∃ L : List Vec4, IsCode L 431 376)
     -- 42 conjuncts
```

The conjuncts are exactly the pairs (n, d) of Tables 2 and 3.

The certificate and its soundness. For each base code C the file contains the column list (encoded as base-q integers and decoded by an explicit function `ofCode`) and the theorem

```
theorem C_431_376_cert : certify C_431_376 431 376 = true := by decide +kernel
```

`certify L n d` checks `L.length = n` and, for each of the $(q^4-1)/(q-1)$ normalized messages (first nonzero coordinate 1), that the support of the codeword has at least d elements. The codeword is computed in *bit-sliced* form. A vector of $\mathbb{F}_q^n$ is stored as 2 bit masks per $\mathbb{F}_3$-component (for $\mathbb{F}_9$) or 3 bit masks (for $\mathbb{F}_8$), and field addition and scalar multiplication become a few `&&&`, `|||`, `^^^` operations on natural numbers, which the Lean kernel evaluates with GMP arithmetic. The soundness theorem

```
theorem isCode_of_certify (L : List Vec4) (n d : Nat) (h : certify L n d = true) :
    IsCode L n d
```

is proved in Lean as follows.

1. Bit-level lemmas (`V3.add_spec`, `V3.smul_spec`, `mk3_spec`, `nz_spec`) show that bit c of the computed masks encodes $u \cdot c_c$.
2. The lemma `countP_eq_pc` shows that the bit count equals `wt (codeword L u)`.
3. A projective reduction (`wt_smul`, using field identities proved by `decide`) extends the bound from normalized messages to all $u \neq 0$.

Puncturing (Lemma 2.2) is the Lean theorem

```
theorem isCode_drop (L : List Vec4) (n d t : Nat) (h : IsCode L n d) :
    IsCode (L.drop t) (n - t) (d - t)
```

and each conjunct of `main9`/`main8` is proved as ⟨`C.drop t, isCode_drop _ _ _ t C_isCode`⟩.

Statement audit.

- `IsCode` quantifies over *all* nonzero u. The codeword is the list of the n dot products, and `wt` counts nonzero entries.
- The length is part of the predicate.
- The coefficient structure is a field with q elements: it is built as `Fin 3 × Fin 3`, respectively 3 bits, and the field axioms are proved.
- The table values $n_{\mathrm{up}}(d)$ are *not* part of the Lean statement. That each pair (n, d) improves the table is a comparison with the web page (copies kept with the data).

The column lists in the Lean file were decoded independently and re-checked by enumeration (see above).

Axioms and resources. The output of `#print axioms`, followed by the resource summary printed by the wrapper script used for the run (file `lean/lean_output.txt`), is:

```
'Q8.GF8_is_field' does not depend on any axioms
'Q8.main8' depends on axioms: [propext, Classical.choice, Quot.sound]
'Q9.GF9_is_field' depends on axioms: [propext]
'Q9.main9' depends on axioms: [propext, Classical.choice, Quot.sound]
leanchk: wall=436.45s maxrss=3865576kB exit=0
```

Only the standard axioms appear. There is no `sorry` and no `native_decide` (no `Lean.ofReduceBool`): the 32 certificates are evaluated by the kernel (`decide +kernel`). The whole file checks in 436 s with a peak memory of 3.9 GB. The option `set_option Elab.async false` is set only to keep the number of threads, and hence the address space, small under the checker's memory limit; it has no logical effect.

5 Limitations and open questions

- *Lower bounds are cited, not re-proved.* Corollary 2.5 depends on [9, Thm. 1.1(4)] and [11, Thm. 1.7(4)]. Only the upper bounds (existence of the codes) are formally verified.
- *Novelty* is asserted relative to Maruta's online table as of October 8, 2026, the papers [9, 11, 1, 12, 10], [4], [15], and a literature search. It is "to our knowledge" only.
- The search is heuristic: the integer programs were usually stopped by a time limit, and only cyclic groups were tried. We did not search $q = 7$ or $k \geq 5$, nor the short-length entries covered by Grassl's tables.
- Most remaining gaps in the mid-range $q = 9$ table are still 2–3 wide. For many targets (e.g. $q = 9$, $67 \leq r \leq 78$, or $q = 8$, $r \in \{48, 49, 58, 59, 60\}$) our best codes fall exactly one point short of the length that would meet the lower bound. It would be interesting to know whether this is a genuine obstruction.
- The 8- and 9-divisible codes found here (Section 3) suggest an infinite family; a geometric construction of them is open.

References

[1] Nanami Bono, Maya Fujii, and Tatsuya Maruta. On optimal linear codes of dimension 4. *Journal of Algebra Combinatorics Discrete Structures and Applications*, 8(2):73–90, 2021.

[2] Michael Braun, Axel Kohnert, and Alfred Wassermann. Optimal linear codes from matrix groups. *IEEE Transactions on Information Theory*, 51(12):4247–4251, 2005.

[3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, 2021.

[4] Markus Grassl. Bounds on the minimum distance of linear codes and quantum codes. Online tables, `https://www.codetables.de`, accessed October 8, 2026, 2026.

[5] J. H. Griesmer. A bound for error-correcting codes. *IBM Journal of Research and Development*, 4:532–542, 1960.

[6] Noboru Hamada. A characterization of some $[n, k, d; q]$-codes meeting the Griesmer bound using a minihyper in a finite projective geometry. *Discrete Mathematics*, 116:229–268, 1993.

[7] Raymond Hill. Optimal linear codes. In C. Mitchell, editor, *Cryptography and Coding II*, pages 75–104. Oxford University Press, 1992.

[8] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018.

[9] Rie Kanazawa and Tatsuya Maruta. On optimal linear codes over $\mathbb{F}_8$. *Electronic Journal of Combinatorics*, 18(1):#P34, 2011.

[10] Shintaro Kawamoto, Atsuya Kato, and Tatsuya Maruta. Locally three-weight property for linear codes and its application. *Serdica Mathematical Journal*, 52:219–248, 2026.

[11] Kazuki Kumegawa, Tsukasa Okazaki, and Tatsuya Maruta. On the minimum length of linear codes over the field of 9 elements. *Electronic Journal of Combinatorics*, 24(1):#P1.50, 2017.

[12] Wen Ma and Jinquan Luo. Nonexistence of some four dimensional linear codes attaining the Griesmer bound. *Advances in Mathematics of Communications*, 18(2):267–282, 2024.

[13] T. Maruta, M. Shinohara, and M. Takenaka. Constructing linear codes from some orbits of projectivities. *Discrete Mathematics*, 308:832–841, 2008.

[14] Tatsuya Maruta. Griesmer bound for linear codes over finite fields. Online tables, `http://mars39.lomo.jp/opu/griesmer.htm` (pages `bound8_4*.htm`, `bound9_4-*.htm`); index page "Last change: 08/09/2026"; accessed October 8, 2026, 2026.

[15] Rudolf Schürer and Wolfgang Ch. Schmid. MinT: the database of optimal net, code, OA, and OOA parameters. `http://mint.sbg.ac.at/`, database version 2024-09-05, accessed October 8, 2026, 2024.

[16] G. Solomon and J. J. Stiffler. Algebraically punctured cyclic codes. *Information and Control*, 8:170–179, 1965.

A All improved table entries

Columns: d; Griesmer bound $g = g_q(4, d)$; the interval n_{lo}–n_{up} in Maruta's table [14] as of October 8, 2026; and the new upper bound. Entries marked * meet the lower bound, so $n_q(4, d)$ is determined (Corollary 2.5). The base code and puncturing amount for every row are listed in `data/improvements.csv`. (For $d = 196$, $q = 9$, the lower bound 222 of the table has meanwhile been raised to 223 by [10, Thm. 1.3(c)]; the entry remains open.)

Table 2: Improved entries of the table of $n_8(4,d)$ (42 entries).

d	g	table	new	d	g	table	new	d	g	table	new
137	159	159–161	160	335	384	384–386	385	363	416	417–418	**417***
138	160	160–162	161	337	387	388–389	**388***	364	417	418–419	**418***
139	161	161–163	162	338	388	389–390	**389***	365	418	419–420	**419***
140	162	162–164	163	339	389	390–391	**390***	366	419	420–421	**420***
221	254	255–257	256	340	390	391–392	**391***	367	420	421–422	**421***
222	255	256–258	257	341	391	392–393	**392***	368	421	422–423	**422***
223	256	257–259	258	342	392	393–394	**393***	369	423	424–425	**424***
224	257	258–260	259	343	393	394–395	**394***	370	424	425–426	**425***
329	378	378–380	379	344	394	395–396	**395***	371	425	426–427	**426***
330	379	379–381	380	345	396	397–398	**397***	372	426	427–428	**427***
331	380	380–382	381	353	405	406–407	**406***	373	427	428–429	**428***
332	381	381–383	382	354	406	407–408	**407***	374	428	429–430	**429***
333	382	382–384	383	361	414	415–416	**415***	375	429	430–431	**430***
334	383	383–385	384	362	415	416–417	**416***	376	430	431–432	**431***

Table 3: Improved entries of the table of $n_9(4,d)$ (149 entries).

d	g	table	new	d	g	table	new	d	g	table	new
190	216	216–219	218	307	347	348–351	350	398	449	450–453	452
191	217	217–220	219	308	348	349–352	351	399	450	451–454	453
192	218	218–221	220	309	349	350–353	352	400	451	452–455	454
194	220	220–224	223	310	350	351–354	353	401	452	453–456	455
195	221	221–225	224	311	351	352–355	354	402	453	454–457	456
196	222	222–226	225	312	352	353–356	355	403	454	455–458	457
197	223	224–227	226	333	376	376–380	379	404	455	456–459	458
198	224	225–228	227	334	378	378–382	380	405	456	457–460	459
199	226	226–229	228	335	379	379–383	381	406	459	459–462	460
200	227	227–230	229	336	380	380–384	382	407	460	460–463	461
231	261	262–265	264	337	381	381–385	384	408	461	461–464	462
232	262	263–266	265	338	382	382–386	385	409	462	462–465	463
241	272	273–276	275	339	383	383–387	386	410	463	463–466	464
242	273	274–277	276	340	384	384–388	387	411	464	464–467	465
243	274	275–278	277	341	385	385–389	388	412	465	465–468	466
253	287	287–290	289	342	386	386–390	389	413	466	466–469	467
254	288	288–291	290	343	388	388–392	390	414	467	467–470	468
255	289	289–292	291	344	389	389–393	392	415	469	469–472	471
256	290	290–293	292	345	390	390–394	393	416	470	470–473	472
262	297	297–300	299	346	391	391–395	394	417	471	471–474	473
263	298	298–301	300	347	392	392–396	395	424	479	479–482	481
264	299	299–302	301	348	393	393–397	396	425	480	480–483	482
265	300	300–303	302	349	394	394–398	397	426	481	481–484	483
266	301	301–304	303	350	395	396–399	398	427	482	482–485	484
267	302	302–306	305	351	396	397–400	399	428	483	483–486	485
268	303	303–307	306	352	398	398–401	400	429	484	484–487	486
269	304	304–308	307	353	399	399–402	401	433	489	490–492	491
270	305	305–309	308	354	400	400–403	402	434	490	491–493	492
271	307	307–310	309	355	401	401–404	403	435	491	492–494	493
272	308	308–311	310	356	402	402–405	404	436	492	493–495	494

Table 3: (continued) Improved entries of the table of $n_9(4, d)$ (149 entries).

d	g	table	new	d	g	table	new	d	g	table	new
273	309	309–312	311	357	403	403–406	405	437	493	494–496	495
274	310	310–314	312	358	404	405–407	406	438	494	495–497	496
275	311	311–315	313	359	405	406–408	407	487	550	550–552	551
276	312	312–316	314	360	406	407–409	408	488	551	551–553	552
277	313	313–317	316	370	418	418–422	420	489	552	552–554	553
278	314	315–318	317	371	419	419–423	421	496	560	560–562	561
279	315	316–319	318	372	420	420–424	422	497	561	561–563	562
289	327	328–331	330	373	421	421–425	424	498	562	562–564	563
290	328	329–332	331	374	422	423–426	425	499	563	563–565	564
291	329	330–333	332	375	423	424–427	426	500	564	564–566	565
292	330	331–334	333	379	428	429–432	431	505	570	570–572	571
298	337	338–341	340	380	429	430–433	432	506	571	571–573	572
299	338	339–342	341	381	430	431–434	433	507	572	572–574	573
300	339	340–343	342	382	431	432–435	434	508	573	573–575	574
301	340	341–344	343	383	432	433–436	435	509	574	574–576	575
302	341	342–345	344	384	433	434–437	436	510	575	575–577	576
303	342	343–347	345	385	434	435–438	437	511	576	576–578	577
304	343	344–348	347	386	435	436–439	438	512	577	578–579	**578***
305	344	345–349	348	387	436	437–440	439	513	578	579–580	**579***
306	345	346–350	349	397	448	449–452	451				

B Data and reproduction

- `data/codes/`: the 32 base codes. The first line of each file is `q= k= n= d=`; each further line `m x0 x1 x2 x3` is a column, repeated m times, in the field encoding of Section 3.
- `data/improvements.csv`: all 191 entries with base code and puncturing amount.
- `data/SHA256SUMS`: hashes of all data and Lean files.
- `lean/Main.lean`, `lean/lean_output.txt`: the formal certificate and its checker output.

To reproduce the verification, run `python3 data/verify.py data/codes/q8_n431_d376.txt` for each code (exhaustive enumeration, seconds per code) and `lean Main.lean` in the directory `lean/` with Lean 4.34.1 ($\approx$ 7 minutes on a dedicated core, up to about twice that on a loaded machine; peak memory about 3.9 GB).